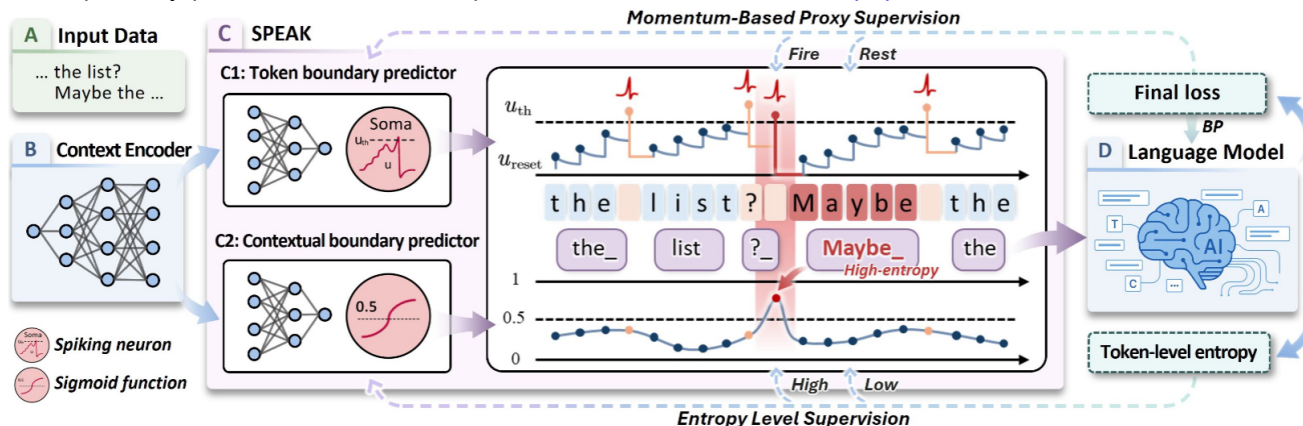


[ACL-2026 Main] SPEAK: Spiking Neurons as an Entropy-Aware Tokenizer for Large Language Models

📖 Introduction

This repository provides the official implementation of the [ACL-2026 paper](#).



Tokenizers are critical for large language models. However, existing methods:

- do not explicitly **leverage historical tokenization results**;
- do not **selectively utilize history** based on contextual relevance.

To address these issues, we propose **SPEAK**, a gradient-based tokenizer that:

- uses **spiking neurons** to model historical tokenization results;
- introduces an **entropy-aware reset mechanism**:
 - **high-entropy tokens** = semantic isolation → hard reset (discard history);
 - **low-entropy tokens** = semantic continuity → soft reset (preserve relevant history).

Experiments on 2 language models and 5 datasets spanning 16 languages demonstrate superior cross-lingual adaptability with competitive performance and efficiency.

🌟 Experimental Frameworks

This repository contains two independent experimental frameworks: gradient-based tokenizer (GTok) and rule-based tokenizer (RTok).

GTok Experiments ([GTok_Experiments/](#))

- **Purpose:** Compare with SOTA GToks (MAGNET, DTP)
- **Model:** Hourglass Transformer trained from the scratch
- **Datasets:** text8 (en), cc-100 (en), wiki40b (en, fi, he, vi)
- **Metrics:** Bits-per-character (BPC) ↓, shortening factor (SF) ↑
- **Result:** BPC 1.108, SF 4.32×

RTok Experiments ([RTok_Experiments/](#))

- **Purpose:** Compare with SOTA RToks (DyTok, ZeTT)
- **Model:** XLM-R (base, 270M) + LoRA
- **Datasets:**
 - XNLI (13 languages)
 - UNER (5 languages)
- **Metrics:** Accuracy / F1 ↑, token sequence length ↓
- **Result:**
 - XNLI: 73.1% (+1.1%)
 - UNER: 79.2% (+1.1%)
 - Efficiency gain: 5.3%–13.4%

Environment Setup

```
# Create and activate conda environment
conda create -n speak python=3.11
conda activate speak

# Install dependencies
pip install -r requirements.txt
```

Run Experiments

```
cd GTok_Experiments # or RTok_Experiments
python main.py
```

All experimental configurations (datasets, hyperparameters, reset modes, etc.) are defined within the respective main.py files. Modify the configuration section in each main.py according to your experimental requirements.

Citation

If you find this code helpful, we would appreciate it if you cite our paper:

```
@inproceedings{chen2026speak,
  title = {{SPEAK}: {Spiking} {Neurons} as an {Entropy}-{Aware} {Tokenizer}
for {Large} {Language} {Models}},
  author = {Chen, Ming and Li, Wenyao and Liang, Chao and Gu, Shi and Lin,
Peng and Ma, De and Tang, Huajin and Zheng, Qian and Pan, Gang},
  booktitle={Proceedings of the 64th Annual Meeting of the Association 748
for Computational Linguistics.},
  year={2026},
  url={https://openreview.net/forum?id=N7hnj8HVDx}
}
```